

P0288R5

any_invocable

Published Proposal, 2019-10-06

This version:

<https://wg21.link/p0288>

Authors:

Ryan McDougall <mcdougall.ryan@gmail.com>

Matt Calabrese <metaprogrammingtheworld@gmail.com>

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes a conservative, move-only equivalent of `std::function`.

Table of Contents

1	Brief History
2	Overview
3	Specification
4	Polls from LEWG San Diego Review (2018)
4.1	Support <code>func()</code> , <code>func() const</code> , <code>func() &&</code>
4.2	Support <code>func() &&</code> only
4.3	Remove <code>target/target_type</code>
4.4	Require more stuff (<code>noexcept</code> , <code>const&&</code> , ...)
4.5	Name Options
5	Polls from LEWG Kona Review (2019)
5.1	We want to spend time on this now in order to include it in C++20
5.2	Add support for <code>func() const&</code> and <code>func()&</code>
5.3	Add support for <code>func() noexcept</code> (x all of the above)
5.4	Include the option for CTAD
5.5	Name: <code>callable</code> vs <code>any invocable</code>
5.6	<code>any_invocable</code> vs <code>invocable</code>
5.7	Header choice
5.8	Can get <code>std::function</code> from <code><any_invocable></code>
5.9	Can get <code>std::function</code> from <code><invocable></code>
5.10	Remove the null-check in the call operator and throwing of <code>bad_function_call</code>
5.11	Remove the null-check in constructors that are not <code>nullptr_t</code>
5.12	Perfect forwarding for converting constructor instead of by-value
5.13	Forward to LWG for C++20
6	Implementation Experience
7	Acknowledgments

§ 1. Brief History

This paper started as a proposal by David Krauss, N4543[1], from 2015 and there has been an open issue in the LEWG bugzilla requesting such a facility since 2014[2].

Since then, the paper has gone through 4 revisions and has been considered in small groups in LEWG multiple times. Gradual feedback has led to the conservative proposal seen here. The most-recent draft prior to this was a late-paper written and presented by Ryan McDougall in LEWGI in San Diego[3]. It included multiple references to implementations of move-only functions and made a strong case for the importance of a move-only form of `std::function`.

Feedback given was encouragement for targeting C++20.

An updated version of that paper was presented on Saturday at the end of the San Diego meeting. Poll results from that presentation are presented after the overview in this document.

The revision was presented in Kona, receiving additional polls and feedback, and was to be forwarded to LWG pending updates reflecting additional poll results. Those changes have been applied to the wording in this paper. Polls from the LEWG Kona review are also provided in this document.

§ 2. Overview

This conservative `any_invocable` is intended to be the same as `std::function`, with the exceptions of the following:

1. It is move-only.
2. It does not have the const-correctness bug of `std::function` detailed in n4348.[4]
3. It provides support for `cv/ref/noexcept` qualified function types.
4. It does not have the `target_type` and `target` accessors (direction requested by users and implementors).
5. Invocation has strong preconditions.

§ 3. Specification

The following is relative to N4820.[5]

Add `<any_invocable>` to [tab:headers.cpp] — C++ library headers

Add the following entry to [tab:support.ft]:

Macro Name	Value	Header(s)
<code>__cpp_lib_any_invocable</code>	<code>xxxxxxL</code>	<code><any_invocable></code>

Make it so that `<functional>` is guaranteed to include `<any_invocable>` by adding the following at the start of the synopsis of [functional.syn]:

```

20.14.1 Header <functional> synopsis [functional.syn]

#include <any_invocable> // see [inv.syn]

namespace std {
```

Insert the following section at the end of **Function Objects** [function.objects], where SECTION is a placeholder for the root of the section numbering:

```

SECTION Storage for any callable [inv]
```

- 1 This subclause describes components that C++ programs may use to perform

operations on callable objects of a discriminated type.

SECTION.1 Header <any_invocable> synopsis

[inv.syn]

- 1 The header <any_invocable> provides partial specializations of any_invocable for each combination of the possible replacements of the placeholders *cv*, *ref*, and *noex* where:

(1.1) – *cv* is either `const` or `empty`.

(1.2) – *ref* is either `&`, `&&`, or `empty`.

(1.3) – *noex* is either `true` or `false`.

- 2 For each of the possible combinations of the placeholders mentioned above, there is a placeholder *inv-quals* defined as follows:

(2.1) – If *ref* is `empty`, let *inv-quals* be *cv&*

(2.2) – otherwise, let *inv-quals* be *cv ref*.

```
namespace std {
    template<class Sig> class any_invocable; // never defined

    template<class R, class... ArgTypes>
        class any_invocable<R(ArgTypes...) cv ref noexcept(noex)>;
}
```

SECTION.2 Class <any_invocable>

[inv.class]

```
namespace std {
    template<class Sig> class any_invocable; // never defined

    template<class R, class... ArgTypes>
        class any_invocable<R(ArgTypes...) cv ref noexcept(noex)> {
        public:
            using result_type = R;

            // SECTION.3, construct/copy/destroy
            any_invocable() noexcept;
            any_invocable(nullptr_t) noexcept;
            any_invocable(any_invocable&&) noexcept;
            template<class F> any_invocable(F&&);

            template<class T, class... Args>
                explicit any_invocable(in_place_type_t<T>, Args&&...);
            template<class T, class U, class... Args>
                explicit any_invocable(in_place_type_t<T>, initializer_list<U>, Args&&...);

            any_invocable& operator=(any_invocable&&);
            any_invocable& operator=(nullptr_t) noexcept;
            template<class F> any_invocable& operator=(F&&);
            template<class F> any_invocable& operator=(reference_wrapper<F>) noexcept;

            ~any_invocable();

            // SECTION.4, any_invocable modifiers
            void swap(any_invocable&) noexcept;

            // SECTION.5, any_invocable capacity
            explicit operator bool() const noexcept;

            // SECTION.6, any_invocable invocation
            R operator()(ArgTypes...) cv ref noexcept(noex);

            // SECTION.7, null pointer comparisons
```

```

friend bool operator==(const any_invocable&, nullptr_t) noexcept;

// SECTION.8, specialized algorithms
friend void swap(any_invocable&, any_invocable&) noexcept;
};
}

```

- 1 The `any_invocable` class template provides polymorphic wrappers that generalize the notion of a callable object (20.14.2). These wrappers can store, move, and call arbitrary callable objects (20.14.2), given a call signature (20.14.2), allowing functions to be first-class objects.
- 2 Implementations are encouraged to avoid the use of dynamically allocated memory for a small contained value. However, any such small-object optimization can only be applied to types `T` for which `is_nothrow_move_constructible_v<T>` is true.

SECTION.3 Constructors and destructor

[inv.con]

```

any_invocable() noexcept;
any_invocable(nullptr_t) noexcept;

```

- 1 *Ensures:* `!*this` is true.

```

any_invocable(any_invocable&& f) noexcept;

```

- 2 *Ensures:* If `!f` is true, `*this` has no target; otherwise, the target of `*this` is equivalent to the target of `f` before the construction, and `f` is in a valid state with an unspecified value.

```

template<class F> any_invocable(F&& f);

```

- 3 Let `VT` be `decay_t<F>`.

- 4 *Constraints:*

- (4.1) `remove_cvref_t<F>` is not the same type as `any_invocable`, and
- (4.2) `remove_cvref_t<F>` is not a specialization of `in_place_type_t`, and
- (4.3) `is_constructible_v<VT, F>` is true, and
- (4.4) `is_invocable_r_v<R, VT inv-quals, ArgTypes...>` is true, and
- (4.5) `!noex || is_nothrow_invocable_r_v<R, VT inv-quals, ArgTypes...>` is true.

- 5 *Expects:* `VT` meets the *Cpp17Destructible* requirements, and if `is_move_constructible_v<VT>` is true, `VT` meets the *Cpp17MoveConstructible* requirements.

- 6 *Ensures:* `!*this` is true if any of the following hold:

- (6.1) `f` is a null function pointer value, or
- (6.2) `f` is a null member pointer value, or
- (6.3) `remove_cvref_t<F>` is a specialization of the `any_invocable` class template, and `!f` is true.

- 7 Otherwise, `*this` targets an object of type `VT` initialized with `std::forward<F>(f)` by *direct-non-list-initialization*.

- 8 *Throws:* Does not throw exceptions when `VT` is a function pointer or a `reference_wrapper<T>` for some `T`. Otherwise, may throw `bad_alloc` or any exception thrown by the expression `VT(std::forward<F>(f))`.

```

template<class T, class... Args>
explicit any_invocable(in_place_type_t<T>, Args&&... args);

```

```

9      Let VT be decay_t<T>.

10     Constraints:

(10.1)      - is_constructible_v<VT, Args...> is true, and
(10.2)      - is_invocable_r_v<R, VT inv-quals, ArgTypes...> is true, and
(10.3)      - !noex || is_nothrow_invocable_r_v<R, VT inv-quals, ArgTypes...> is true.

11     Mandates: VT is the same type as T.

12     Expects: VT meets the Cpp17Destructible requirements, and if
is_move_constructible_v<VT> is true, VT meets the Cpp17MoveConstructible
requirements.

13     Ensures: *this targets an object of type VT initialized with
std::forward<Args>(args)... by direct-non-list-initialization.

template<class T, class U, class... Args>
    explicit any_invocable(in_place_type_t<T>, initializer_list<U> ilist, Args&&... args);

14     Let VT be decay_t<T>.

15     Constraints:

(15.1)      - is_constructible_v<VT, initializer_list<U>&, ArgTypes...> is true, and
(15.2)      - is_invocable_r_v<R, VT inv-quals, ArgTypes...> is true, and
(15.3)      - !noex || is_nothrow_invocable_r_v<R, VT inv-quals, ArgTypes...> is true.

16     Mandates: VT is the same type as T.

17     Expects: VT meets the Cpp17Destructible requirements, and if
is_move_constructible_v<VT> is true, VT meets the Cpp17MoveConstructible
requirements.

18     Ensures: *this targets an object of type VT initialized
with ilist, std::forward<ArgTypes>(args)... by direct-non-list-initialization.

any_invocable& operator=(any_invocable&& f);

19     Effects: Replaces the target object of *this with the target object of f.

20     Ensures: If !f is true, *this has no target; otherwise, the target of *this
is equivalent to the target of f before the construction, and f is in a
valid state with an unspecified value.

21     Returns: *this.

any_invocable& operator=(nullptr_t) noexcept;

22     Effects: If *this != nullptr is true, destroys the target object of this.

23     Ensures: !*this is true.

24     Returns: *this.

template<class F> any_invocable& operator=(F&& f);

25     Effects: Equivalent to:
        any_invocable(std::forward<F>(f)).swap(*this);
        return *this;

template<class F> any_invocable& operator=(reference_wrapper<F> f) noexcept;

```

```

26      Effects: Equivalent to:
          any_invocable(f).swap(*this);
          return *this;

~any_invocable();

27      Effects: If *this != nullptr is true, destroys the target object of this.

SECTION.4 Modifiers [inv.mod]

void swap(any_invocable& other) noexcept;

1      Effects: Exchanges the targets of *this and other.

SECTION.5 Capacity [inv.cap]

explicit operator bool() const noexcept;

1      Returns: true if *this has a target, otherwise false.

SECTION.6 Invocation [inv.inv]

R operator()(ArgTypes... args) cv ref noexcept(noex);

1      Expects: bool(*this) is true.

2      Effects: Equivalent to:
          return INVOKE<R>(static_cast<F inv-quals>(f), std::forward<ArgTypes>(args)...);,
          where F is the type of the target object of *this and f is the target object
          of *this.

SECTION.7 Null pointer comparison functions [inv.nullptr]

bool operator==(const any_invocable& f, nullptr_t) noexcept;

1      Returns: !f.

SECTION.8 Specialized algorithms [inv.alg]

void swap(any_invocable& f1, any_invocable& f2) noexcept;

1      Effects: Equivalent to: f1.swap(f2).

```

§ 4. Polls from LEWG San Diego Review (2018)

§ 4.1. Support func(), func() const, func() &&

```

SF F N A SA
6 6 2 1 0

```

§ 4.2. Support func() && only

```

SF F N A SA
2 2 7 1 1

```

§ 4.3. Remove target/target_type

```
SF F N A SA
12 5 0 0 0
```

4.4. Require more stuff (noexcept, const&&, ...)

```
SF F N A SA
0 1 8 6 0
```

Note that the final poll (require more stuff) was not due to members being against the design, but because we could easily add those facilities in a later standard without any breakage.

4.5. Name Options

There was one final poll, which brought us to the name `any_invocable` .

```
3 unique_function
3 move_function
2 move_only_function
7 movable_function
8 mfunction
10 any_invocable
8 mofun
8 mofunc
0 callback
4 mvfunction
2 func
0 callable
2 any_function
```

5. Polls from LEWG Kona Review (2019)

5.1. We want to spend time on this now in order to include it in C++20

```
SF F N A SA
8 8 2 0 0
```

5.2. Add support for `func() const&` and `func()&`

```
SF F N A SA
0 8 7 0 0
```

5.3. Add support for `func() noexcept` (x all of the above)

```
SF F N A SA
2 12 2 0 0
```

5.4. Include the option for CTAD

```
SF F N A SA
0 1 5 9 0
```

5.5. Name: callable vs any invocable

```
SC C N AI SAI
3 2 3 5 6
```

5.6. any_invocable vs invocable

```
SAI AI N I SI
3 7 2 5 1
```

5.7. Header choice

```
7 <functional>
11 <any_invocable>
11 <invocable>
3 <()>
```

5.8. Can get std::function from <any_invocable>

```
SF F N A SA
0 1 4 4 7
```

5.9. Can get std::function from <invocable>

```
SF F N A SA
1 3 6 3 2
```

Decide on <any_invocable>. Unanimous for <functional> to pull it in, even if in its own header.

5.10. Remove the null-check in the call operator and throwing of bad_function_call

```
SF F N A SA
8 2 1 0 0
```

5.11. Remove the null-check in constructors that are not nullptr_t

```
std::any_callable ac = my_ptr_object;
if(ac) { /* true even if my_ptr is nullptr */ }
```

```
SF F N A SA
0 2 2 4 3
```

5.12. Perfect forwarding for converting constructor instead of by-value

Unanimous

5.13. Forward to LWG for C++20

```
SF F N A SA
8 5 0 0 0
```


§ 6. Implementation Experience

There are many implementations of a move-only `std::function` with a design that is similar to this. What is presented is a conservative subset of those implementations. The changes suggested in LEWG, though minimal, have not been used in a large codebase.

Previous revisions of this paper have included publicly accessible move-only function implementations, notably including implementations in HPX, Folly, and LLVM.

§ 7. Acknowledgments

Thanks to Tomasz Kamiński, Tim Song, and Nevin Liber for suggestions on wording simplifications.

§ 8. References

[1]: David Krauss: N4543 "A polymorphic wrapper for all Callable objects" <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4543.pdf>

[2]: Geoffrey Romer: "Bug 34 - Need type-erased wrappers for move-only callable objects" https://issues.isocpp.org/show_bug.cgi?id=34

[3]: Ryan McDougall: P0288R2 "The Need for `std::unique_function`" <https://wg21.link/p0288r2>

[4]: Geoffrey Romer: N4348 "Making `std::function` safe for concurrency" www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4348.html

[5]: Richard Smith: N4820 "Working Draft, Standard for Programming Language C++" <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4820.pdf>